

3 pieces: Virtualization, Concurrency, persistence

Lecture 2 OS Basis

2.1 Computer: arithmetic/logic operations automatically

(1) How do modern C works?

von Neumann: single, shared MEM for programs and data, single bus for mem access, an arithmetic unit, a program control unit
Hardware, OS, Application programs, Users

(2) OS

Def: software operate com efficiently correctly
Exe user program
Make system ez to use
Include kernel, manage all physical devices, expose sys call; helper program(shell, GUI, browser)
Use hardware in efficient manner (hardware abstraction)
Resource manager: hardware, resolve conflicts for resources use.
Control program: prevents error and improper use

(2) usage:

(i) Virtualization:

CPU: Multiple programs on 1 CPU (as if multiple CPUs)
Mem: Multiple process on same Mem, as if in its own

(ii) Concurrency: Run multi-threaded programs and exe correctly

(iii) Persistence: write data from volatile to persistent storage

(3) History

2.2 Dual-mode Operations

(1) Kernel: a bigger library to handle low-level I/O

↳ dual mode: kernel, user mode; some instructions designated

Mode bits provided by CPU, distinguish sys mode

Privileged instructions only in kernel mode/fail or trap

U → K 存 PC, user state, 跳 K → U 恢复, 回 user mode, 继续

x86 Ring 3 ... 0: 3 app, 0 kernel

RISC-V: User app < Supervisor kernel < Machine firmware

(2) Mode transition:

① System Call: Process request system service, function call across
Does not have the address of system function to call, privilege

Marshall the syscall id and args in reg → exec syscall

e.g. read(rfd, buf, size): User → syscall → kernel → disk → ret

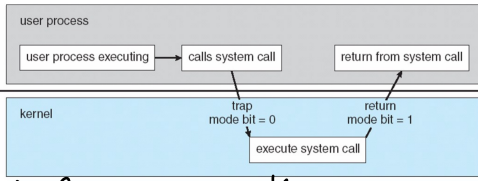
② Interrupt: external asynchronous event triggers context switch

Timer/I/O, independent of user process, 外部打断, 不可控

③ Trap/Exception: internal synchronous in process triggers context switch

Trap (如: 溢出, 除数异常) e.g. syscall | Exception: protect violation

CPU 过程: 停 → 存当前 → 查表 ID → handler → 处理 → 返回



a number associate each syscall, table index
Details hidden by lib API

User service: Communication, err detect
UI, Program exe, I/O, File-sys man, System: Resource Allocate, Accounting, Protection & Security

2.3 Kernel structure

(1) Taxonomy: ① Monolithic: everything in kernel, fast, easy to write, spot, hard to maintain, not stable

② Micro: only core in kernel, IPC, Scheduler, Virtual Mem, other as user level servers (trusted), Protection in kernel safer, stable, module, slow, complex resource management policy go to user level servers

(2) Virtualization and Hypervisors
Hypervisor (virtual machine manager) emphasizes on virtualization and isolation
OS can run on hypervisor almost without modification resource partition

(3) OS design principle

User goal ↔ System goal | Policy ↔ Mechanism (How is policy enforced)

2.4 Operating System Services

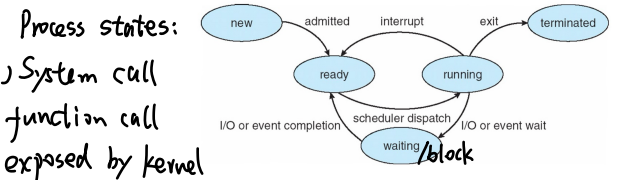
User interface, Program exe, I/O, File system manipulation
Communications, Err detection, resource alloc, accounting, Protection
System Programs: file manage, modify, status info, programming language support, program loading and execution, Communications (processes, users, computers)

Lecture 3 Processes

3.1 Process and System Calls

(1) Process: a program in execution, program is a file on the disk
process is loaded by OS, code and static data core are loaded heap and stack created by OS
Reg: I/O
Abstraction of machine states: Mem addr space, PC, Stack, frame.

(2) Process identification: unique ID number PID, sys call getpid()



(3) System call

function call exposed by kernel

abstraction of kernel operations: is a call by number

Parameter passing:

registers: x86, risc-V

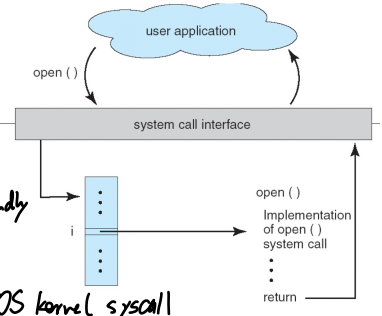
blocks: in mem, pass addr

stack: [para] pop OS

SysCall: primitive, not programmer-friendly

Lib API call: invokes syscall

APP invoke lib, lib implement syscall, OS kernel syscall



3.2 Process Creation

(1) Creation: tree of processes, child duplicate parent addr space

(2) fork(): from fork(), before: the same, after: exe the same program

ret = { PID of child: parent process 先后运行
0: child process (running, ready) → (stop, running) }

fork cloning parent: PC, code, mem, opened files
CPU reg file mem kernel's internal

ret value, PID, parent process, running time, file locks different

(3) exec(): execl, execl, execlp, execv ...

exec 替换当前 code, 内存 (全局变量, 局部变量, malloc), reg, PC
不变 kernel: PID, parent, child, open file descriptors, process state

Not return, just exit(), do not go back

(4) wait(): fork 后 顺序不定, parent → fork() → child running wait

wait(): V child terminate, ret 返回, if a child terminates before wait parent ← child exit
waitpid(): particular child, diff status change (resume, stop by signal)

Shell: bash → fork() → parent → wait() → ...
child → exec("ps") → ps ↑ continue

3.3 Processes: kernel view

- (1) PCB: process control block: state, PC, regs, scheduling, memory manage
 (2) Ready queue, I/O device queue
 ready: all processes in ready state
 device: processes waiting for I/O events
 processes migrate
 Accounting, I/O status
 Multiprogram: OS manage
 But moving data in/out of memory is slow
 ⇒ Memory partition: leave processes in memory when switching.
 each process owns a small part of physical mem
 Problem: 需要更多内存? 如何知道自己地址当编译? 越界写内存? (abstraction)
- 4.2 Address space: process' view of mem in computer system
 32 bit CPU → 2^{32} 4GB 0KB
 64 bit CPU → usually 2^{48} 256TB 1KB
- 4.3 Mem Virtualization
 abstraction of a private large addr space
 for multiple running processes
 on top of a single physical mem
 1B = 8 bit Mem addr is of a BYTE
 0x8C 0x8d 0x8e 0x8f
 d1 4A F5 83
 Big endian: 0x d1 4a f5 83 15KB
 Little endian: 0x 83 f5 4a d1 16KB
- 4.4 Addr Translation
 MMU in CPU translate VA used by instructions → PA understood by DRAM
 CPU interposes every memory access; OS setup hardware, keep track free mem
 Fetch ins at VA 0x0bad0000 → 0x00000000 maintain control of how mem is used
 ret with ins at PA 0x00000000 CPU exe ins and access VA 0x12345678
 Data from 0x13170000 go to reg a1
- How to translate?
 (1) Base & Bounds: Dynamic relocation
 2 hardware registers: base register + limit registers
 Exe VA → (limit reg) → (base reg) → PA → DRAM
 Hardware: Privileged operation
 regs: pair of reg per CPU
 OS Mem Manage: reclaim mem from zombie mem free list
 Base Bounds: set base/bounds upon when Content switch
 Privileged instruction to register exception handlers: Exception: Code to run exception
 Limits: Internal fragmentation: wasted mem between heap & stack
 Cannot support larger addr space: which = allocated slot in mem
 Hard to do inter-process sharing: no sharing memory(lib)
- (2) Segmentation: 把一个 process 的 code, heap, stack 放在 PA 的不连续地方
 segment 表: seg ID#, Base, Bounds ⇒ VA = [seg ID][offset]
 每次用找表, 取 base, bounds, 看 offset <? Bounds 是 yes → PA = Base + offset
 支持 code sharing (base sharing), Need protection, Code Read-exe other Read-Write
 Problems: Context switch need to save all pairs of segment regs
 segment grow may be blocked; Manage of free spaces of PA with external fragmentation: free gaps between allocated segs, also internal if more space all of them need
- (3) Mem Allocation: ideal: fast, minimizes fragmentation
 Basic: linked list of free slots
 Strategy: ① Best fit: 找够用的最小; 浪费小, 遍历很慢; 小碎片
 ② Worst fit: 找最大, 留下较大的连续空间, 遍历很慢; 碎片更多
 ③ First fit: 找到够用的, 快, 前面堆碎片
 No best option
- 3.4 fork(), exec(), wait() Kernel view
 (1) fork(): syscall, user space → kernel space, new process, new PCB
 复制 addr space, file page, PC, reg, 加入 task list (双亲有孩子)
 对父, 更新 list of children; 对子: 更新 pid, running time → 0, pointer to parent
 复制 ① Duplicate Addr space: 完整复制父内存
 ② CoW: - 开始共享, 只读, 当触发写, 复制该页给特定 process
 Opened files: 共享文件 offset (读写指针), 同时写会乱序
 (2) exec(): 换皮不换骨 → 留 PCB entry 直到 parent call wait()
 (3) exit(): free all allocated memory, close opened file list
 then frees everything on user-space mem, 进程已死
 (terminate) zombie, pid remains, in process table, send SIGCHLD to parent
 (4) wait(): register signal handling routine → invoke, then
 ① accept, remove the SIGCHLD signal; ② destroy child p (p table task list)
 ③ deregister sig handle routine ④ return pid of 1st child as wait()
 Normal case: Parent wait → child exit 正常结束
 P.W.I after C exit(): until wait() called, die immediately
 important because pid max = 32768 < default
- 3.5 More on processes
 (1) init: pid=1, parent of all, fork() and exec()
 (2) process tree: each process has parent and children (pstree)
 (3) Orphan: 孤儿, re-parenting: when exit, all children parent → init
 (4) Bg jobs: shell 退, program 变 orphan, 仍在 init 下运行
 (5) Measure Process time: real, user, sys
 real = user + sys > I/O intensive I/O or syscall is expensive
 < multi-core
- 3.6 Threads: abstraction of exe of a program: n points of execution
 (Context switch) each thread has its own private execution state: PC, reg, stack
 threads in the same process share addr space, files, signal...
 并行: 减少 I/O 阻塞, allow sharing resources. Computing resources
 User thread: thread management done by user-level threads lib. OS don't know
 Kernel thread: done by kernel, is aware of each kernel thread
 thread model: M-to-M 要 threads 在 user 和 kernel 都 implement
 M-to-1: context switch ~ threads cheap; 1 thread block I/O, all block
 1-to-1: each run or block independent; need to → kernel to schedule
 M-to-M: n user + sn kernel threads; flexible, complex

Lecture 4 Address Translation

4.1 Multiprogramming

Early: set of lib uses lower mem, 1 running program uses rest of mem

(3) When to trigger page replacement
 proactive → better performance: replace even no one needs free yet
 always reserve free page frame
 Swap daemon: high, Daemon sleep 5 ms 1/5 1/5 1/5 1/5
 low, Daemon wake 1 1/3 1/6 1/1

6.2 Replacement Policy: Effective Access Time = hit% × T_{hit} + Miss% × T_{miss} (hit% = 1/1.5, miss% = 1/1.5)
 (1) 3 miss: ① Compulsory: 冷启动 → pre-jetch; ② Capacity: no enough mem 1/10 DRAM 分配不同 process (1/10)

③ Conflict: fully-associative cache does not have
 (2) Policy: OPT, FIFO, RANDOM, LRU, LFU (u = stack algo)
 Bélady Anomaly: FIFO → more page frames may more page fault
 (3) LRU implementation 4GB 有 1M page frame
 Why no true: 硬件支持 timestamp, 访问时开销, TLB 也不解决, 排序太慢
 ① Ref bit: 每 page 1 bit, 用过 1, 0 5 定期置 0
 ② Clock: A range in circular list, sets ref to 1 upon first access
 a pointer, when replace, check ref, if 1: → 0, next; if 0: replace, move this out
 ③ Clock with Dirty bit: (ref, dirty) (0,0) > (0,1) > (1,0) > (1,1)
 ④ Additional ref bit: 每个 page 有 1 个 8 bit ref, time interrupt every 1/10 ms, ref bit + history 8 bit > 1, compare as unsigned int, 看 1 的个数
 ⑤ Clock + Counter: when ref = 0 cnt++, if cnt > N, 换

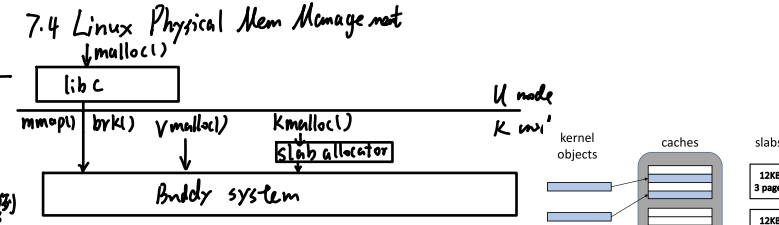
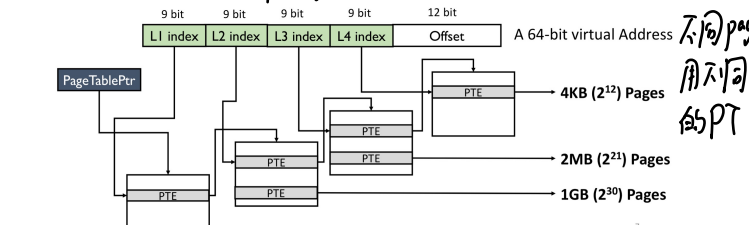
6.3 Page Frame Allocation: 每个进程多少 page frames
 (1) Minimum # of pages/process: 取决于电脑架构, 进程跑 ins 要几页? ins mem page table
 Maximum 取决于总内存
 (2) Global: 所有 frame 共享池, 按认为的 frame (利用率高, 负载均衡, 不公平, 抖动)
 Local: 每个进程只在自己的 frame 里换 (公平, 效率高, 浪费空间)
 (3) Algo: ① 均分: frames/process ② Proportional: S_i = Size of process P_i, m = total # frame
 a_i = m × S_i / S; ③ Priority: 高低 page fault 轮优先级 (text, data, heap, stack, PT)
 (4) Thrashing: CPU 大部分时间在换 page ← working set demands > Physical mem
 Early: 减缓进程数 working set 最近用的一组 page
 Modern: OOM killer 杀 oversubscribed 进程, reboot

Lecture 7 Linux Mem Management

7.1 Addr space in Linux
 split 0-BFFFFFFF user
 0xC0000000-0xFFFFFFFF kernel
 Why kernel mem mapped into addr space of each process
 ① trap 不需换 PT - no TLB flush
 ② kernel 可直接访问 user mem
 Kernel mem in each addr space → SAME
 0xC0000000

7.2 User space and kernel space
 物理连续
 Kernel logic addr: 存 PT, per-process kernel stack, kmalloc() never swap-out
 always map → continuous PA from 0x0, ez for DMA that 要连续物理地址
 kernel virtual addr: 虚拟连续 vmalloc() permission bit
 process Isolation: 不同地址空间; user iso: page table U bit = 1 user mode may access this

7.3 Large Page support: x86 support 4KB, 2MB, 1GB pages
 hardware enforce alignment low 12, 21, 30 bit = 0
 Fewer TLB misses, App may need 连续物理内存, 导致 internal fragmentation

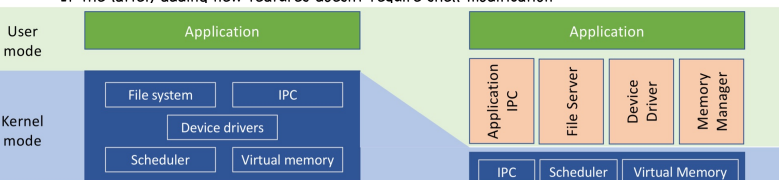


7.4 Linux Physical Mem Management
 (1) define: malloc 可以调用 brk 扩展 heap, mmap 映射文件
 Kmalloc: 物连, 小, 快 (k data, DMA buffer); vmalloc: 虚连, 大, no DMA
 (2) Slab Allocator 专门给固定类型的内核对象准备的预分配内存池
 Consists of 1 + physical contiguous pages, cache consists 1 + slabs: 每种 kobj, 1 种 cache
 empty, partial, full 先 partial, 再 empty, 再 new from buddy
 when slab allocated into a cache, obj are init and marked as free
 ① No mem wasted due to fragmentation: 清闲时, 返回一样多的内存, 内存时排列号
 ② Mem request can be satisfied quickly: 提前预分配 ptr; freed obj 直接标 free
 (3) Buddy System: 管理 1 个 2^n 连续块, 找不到就往上借, 借了再分裂
 X KB → 要 ceil(X/4KB) 连续块, 从下往上取至 1 2^n 的空闲块
 Free: the freed block is recursively merged with its buddy
 Buddy 在同一层, 一个 size. xxx 的 buddy addr = addr XOR block size

7.5 Page Cache: RAM 缓存 disk 的地方存 ① mem-mapped files: binary ② Anonymous mem: stack heap swap space
 tracks it entries are clean/dirty, pdflush 周期写回 dirty pages
 2Q replacement: (LRU 会挤掉真正要用的)
 2 list inactive, active list: 第一次访问 → inactive, 第二次 → active
 ③ 淘汰: 太 inactive, 周期性 move bottom active list → inactive, keeping active list 2/3 size of page cache
 Clock 模拟 LRU 第一次 minor, 淘汰出回写
 Clock 总结: 指针指向最久不使用的, 当 hit 时, ref = 1; 按完指针
 当 page fault 时, if ref = 1, ref = 0 指针 ++; if ref = 0, swap-out

7.6 DEP data exe prevention
 Buffer overflow, attacker 输入超长数据, 溢出栈, 覆盖返回地址 RA
 DEP → WX bit in PTE, stack not executable
 ASLR addr space layout Randomization → stack heap lib c 地址随机
 Return-to-libc: libc 有 system(), 覆盖地址跳到 system()
 Return-Oriented-Prg: 把 libc 代码 snippets 拼出任意功能
 e.g. VA: 0x123456789, page size: 4KB → offset 12 bit
 satp, PPN = 0x/100 根目录, root = 0x/00000000 12: 0x/00000000 VPN 2 → VPN 1 → VPN 0
 VPN 0 1 2 : VA >> 12 21 30 & 1FF (32/64 bit) V = 1, RWX = 0 非 0
 从 VPN 2 开始, pte.addr = root + VPN[x] × PTE size = 0x/0012 14 R = 1/X = 1/4
 User-friendly desktop metaphor interface
 • Usually mouse, keyboard, and monitor
 • Icons represent files, programs, actions, etc
 • Various mouse buttons over objects in the interface cause various actions: provide information, options, execute function, open directory (known as a folder)
 • Invented at Xerox PARC

Many systems now include both CLI and GUI interfaces
 • Microsoft Windows is GUI with CLI "command" shell
 • Apple Mac OS X as "Aqua" GUI interface with UNIX kernel underneath and shells available
 • Solaris is CLI with optional GUI interfaces (Java Desktop, KDE)
 Command Line Interface (CLI) or command interpreter allows direct command entry
 • Sometimes implemented in kernel, sometimes by systems program
 • Shells: Bourne shell, C Shell, Bourne-Again Shell, Korn Shell
 • Primarily fetches a command from user and executes it
 • Sometimes commands built-in, sometimes just names of programs
 • If the latter, adding new features doesn't require shell modification



Lecture 8 CPU Scheduling

8.1 Definition: 1个CPU跑多个 process, 让谁跑

CPU-bound process: 大部分计算, 少用 I/O: user time > sys time

I/O-bound process: 经常等 I/O, CPU 用得少: sys time > user time

CPU burst: process 连续使用 CPU 做计算的一段时间 (大部分时间)

When schedule?

① running → waiting 开始等 I/O ② running → ready 被抢占

③ waiting → ready I/O 完成; ④ terminate 结束

Non-preemptive 只有 ① ④ 调度; Preemptive 主动停, 让别人跑

Opt Criteria: Given: $\{p_1, p_2, \dots\}$ with arrival time, CPU requirement

Minimizing: ① avg Turnaround time: $T_F - T_A$ ② avg waiting time = $\sum T_{ready}$

③ # Context switch (少开销)

8.2 Methods:

(1) SJF: Shortest Job First 让 CPU burst 最短先跑

① Non-preemptive: FIFO break tie

② Preemptive: 每有新 process 到达, 比较, 选最短先跑

(2) Round-Robin: 给每个 process 一个 quantum, 跑完后强制放在队尾, 新 process 不会感觉某 process job "froze", no starvation

(3) Priority Scheduling: 给每个 process 一个 priority, scheduler 选最小的 SJF 是特例, priority = next CPU burst; 静态, 动态优先级 → solve starvation

低优先级 process 可能永远等不到 ← Aging: 随时间推移, priority ↑

8.3 Linux Schedule: Before kernel 2.5, tradition Unix sch, X support SMP

Linux kernel 2.5: O(1) scheduler, 时间是常数, 不随 P 增加, SMP 封装, Linux 2.6.23 引入 CFS

CFS: Complete Fairness Scheduler: 公平分享 CPU, 不是固定 quantum

Nice: -20 ~ +19, 默认 0, Nice ↓ relative priority ↑ Higher value being nice

Task with lower nice receives higher % of CPU Time

Virtual run time (per-task): Decay Factor, priority ↓ decay rate ↑ nice = 0, vrt = physical run time

nice > 0 (low pri), vrt 增长更快 "算多了", vrt > prt

永远选 vrt 最小的执行, 高 pri 可抢占低 pri

nice → vrt 增长速度 → vrt 小 pri 高 → 抢占

Lecture 9 Synchronization

9.1 Race Condition: 同时读, 都是 Mem → reg → mem, 还没写回 CW

"Shared obj" "multiple process/thread" "concurrently" outcome depends on order of shared res access

9.2 Solution: Mutual Exclusion 互斥

当一个 p 在访问共享对象时, 不允许其他 p 同时访问

划定 critical section: 访问共享对象的代码: 进入前上锁, 离开后解锁

当一个 p 在 ↑ 时, 其他想进入 critical section 的 p 只能等待

临界区要尽量短, 范围 ↑, 阻塞其他进程的根效率 ↓

① 互斥性: 没有 2 个 process 可以同时 go in cri section
② 有界等待: 1 个进程想进 cri, 等待次数有上限, 不能一直等
③ 进展性: 如果当前无人在 cri, 想进入的 p 必须直接进

9.3 Implementation Mutual exclusion, Bounded waiting, Progress

(1) Disable interrupt: while process is inside the cri, no other

Unicore: correct but not permissible { user level: 不循环, kernel 不困

Multi-core: incorrect 其他 core 不累加句

(2) Locks can also race condition

Atomic instruction: instructions that cannot be "interrupted"

Not even by instruction running on other core

① Spin-based locks: loop on a shared obj, turn, to detect status of other process, 执行顺序是强制交替: 0-1-2, 1-2 alternating

OK for short wait (spin-time < context-switch-overhead)

Problem: when P1 is busy, violate progress req. P0 进不去, 轮到谁 谁想进 因为交替

Petersen's Solution: 用 2 变量 turn, interested[]

lock: interested[P] = True;

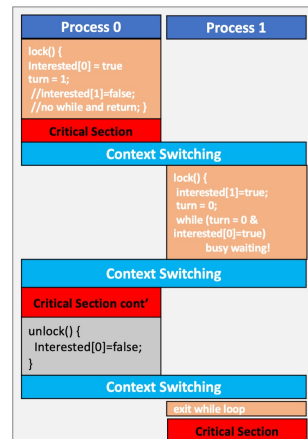
turn = other;

while (turn == other && interested[other] == True);

unlock: interested[P] = False.

Petersen's Solution

```
1 int turn;
2 int interested[2] = {FALSE, FALSE};
3
4 void lock( int process ) {
5     int other;
6     other = 1-process;
7     interested[process] = TRUE;
8     turn = other;
9     while ( turn == other &&
10            interested[other] == TRUE )
11         ; /* busy waiting */
12 }
13 void unlock( int process ) {
14     interested[process] = FALSE;
15 }
```



ME: interested[0] == interested[1] == true | turn = 0/1, not both
Progress: only P0, interested[1] = true | both P0, P1, turn = 0/1, not both

Bounded-waiting: P0 调 lock → turn = 1 让给 P1 → 有限次等待

Multi-Process Mutual Exclusion 多 process

do {

```
waiting[i] = TRUE;
key = TRUE;
while (waiting[i] && key)
    key = test_and_set(&key);
waiting[i] = FALSE;
// critical section
```

```
j = (i + 1) % n;
while ((j != i) && !waiting[j])
    j = (j + 1) % n;
if (j == i)
    lock = FALSE;
else
    waiting[j] = FALSE;
// remainder section
```

} while (TRUE);

Priority inversion:

高 pri 在低 pri 在 cri 中时抢占,

导致高 pri 用 CPU 什么都不干, 低 pri 无法 release lock

② Sleep-based lock: Semaphore initial s value

value: # of available resources; a wait-list / Disable interrupts
must involve kernel, implement 不可打断

Entry:

Void sem_wait(semaphore *s) {

s->val = s->val - 1;

if (s->val < 0) sleep;

}

Exit

Void sem_post(semaphore *s) {

s->val = s->val + 1;

if (s->val <= 0) wakeup();

}

while (1) { sem_wait(s); cri(); sem_post(s); }

a process enter cri if

waiting[i] = false or key = false

Key is local variable

all p must exe test_and_set()

1st call t.a.s with lock = false

key = false

lock = true, after 1st exe t.a.s

key = true 直接跑 t.a.s, 抢到把 lock 设

退出时, 循环扫 waiting, true

找下一个 wait, 直接把它 waiting

如无人等 → lock = false → false

Semaphore implementation

* S.value also shared $\rightarrow$ sem_wait sem_post 内也是 cri, 同一时间只能有 1 process 执行

单核: 禁用中断 多核: Atomic.ins, Cmp_xchg.

Cmp_xchg Compare and exchange Compare.And.Swap.

int Cmp_xchg(int *addr, int expected_value, int new_value)

int temp = *addr; if (*addr == expected_value) *addr = new_value;

return temp; } 读出 addr 当前值, 如是我期望的, 改成新值;

只返回旧值

Atomic_inc(addr) : do { int old = *addr; int new = old + 1;

while (Cmp_xchg(addr, old, new) != old)

9.4 Producer-Consumer Problem (Bounded buffer)

avoid race { Mutual exclusion with one binary semaphore (buffer shared)

sync { Synchronization with two counting semaphore Notify pro full

不能只用一个, mutex 顺序不能换 $\rightarrow$ deadlock, 等着就 sleep Notify can emp

```
Shared object
#define N 100
semaphore mutex = 1;
semaphore avail = N;
semaphore fill = 0;
```

Note

The size of the bounded buffer is "N".

fill : number of occupied slots in buffer
avail: number of empty slots in buffer

Abstraction of semaphore as integer!

```
Producer function
1 void producer(void) {
2   int item;
3
4   while(TRUE) {
5     item = produce_item();
6     wait(&avail);
7     wait(&mutex);
8     insert_item(item);
9     post(&mutex);
10    post(&fill);
11  }
12 }
```

```
Consumer Function
1 void consumer(void) {
2   int item;
3
4   while(TRUE) {
5     wait(&fill);
6     wait(&mutex);
7     item = remove_item();
8     post(&mutex);
9     post(&avail);
10    //consume the item;
11  }
12 }
```

9.5 Dining Philosopher Problem

```
Shared object
#define N 5
#define LEFT ((i+N-1) % N)
#define RIGHT ((i+1) % N)

int state[N];
semaphore mutex = 1;
semaphore p[N] = 0;
```

```
Section entry
1 void take_chopsticks(int i) {
2   wait(&mutex);
3   state[i] = HUNGRY;
4   captain(i);
5   post(&mutex);
6   wait(&p[i]);
7 }
```

```
Main function
1 void philosopher(int i) {
2   think();
3   take_chopsticks(i);
4   eat();
5   put_chopsticks(i);
6 }
```

```
Section exit
1 void put_chopsticks(int i) {
2   wait(&mutex);
3   state[i] = THINKING;
4   captain(LEFT);
5   captain(RIGHT);
6   post(&mutex);
7 }
```

```
void wait(semaphore *s) {
  *s = *s - 1;
  if (*s < 0) {
    sleep();
  }
}
```

```
void post(semaphore *s) {
  *s = *s + 1;
  if (*s <= 0)
    wakeup();
}
```

Extremely important helper function

```
1 void captain(int i) {
2   if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
3     state[i] = EATING;
4     post(&p[i]);
5   }
6 }
```

CS334 Operating Systems (H)

Lecture 10 DeadLock

10.1 Definition: deadlock $\rightarrow$ starvation $\times$ deadlock

Mutual Exclusion: only 1 thread at a time can use a resource

Hold and wait: thread hold resource, wait to acquire additional res held by other

No preemption: resource can only be released voluntarily

Circular wait: Always exists a set $\{T_1, \dots, T_n\}$ $T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_n \rightarrow T_1$

10.2 Resource-Allocation Graph $\rightarrow$ CPU cycles, mem space, I/O

Set of thread: T_n , Resource types: R_m , W_i instances, t utilize r as request (1 use)

(t, R) $T_i \rightarrow R_j$ request edge

$R_j \rightarrow T_i$ assignment edge

10.3 methods for handling deadlocks:

(1) Allow, then recover: detect
No cycle in RAG $\rightarrow$ No deadlock
if cycle: each R, 1 instance $\rightarrow$ deadlock

if multiple instances,

Available: length m, each resource has ? 可用的 instance (0, 0, 0)

Allocation: nxm matrix, # of res $\rightarrow$ each process $P_0 = (0, 1, 0) \dots$

Request: nxm matrix, current request of each process $Req[i, j] = k$

* Most OS ignore, pretend dl never happen.

P_i req k more ins of R_j

1. Let Work and Finish be vectors of length m and n, respectively Initialize:

(a) Work = Available

(b) For $i = 1, 2, \dots, n$, if $Allocation_i \neq 0$, then

Finish[i] = false; otherwise, Finish[i] = true

2. Find an index i such that both:

(a) Finish[i] == false

(b) Request_i $\leq$ Work

3. Work = Work + Allocation;
Finish[i] = true
go to step 2

4. If Finish[i] == false, for some $i, 1 \leq i \leq n$, then the system is in deadlock state. Moreover, if Finish[i] == false, then P_i is deadlocked

If no such i exists, go to step 4

$\Rightarrow$ 安全序列

Then terminate process: not always possible

② preempt res: take away res temporarily, does not always fit with semantics of computation

③ Roll back actions of dled process: may again.

10.4 DL Prevention

XME: X some are shared. utilization $\downarrow$ may starve.

X H-A-W: 必须保证申请 res 时, 不 hold 其他; 且一开就立刻申请全部 (在天的时候)

X No preemption: 如申请失败, 则释放所有 res held, 等条件满足 together 一起再问

X Circular wait: impose a total ordering of all Res, must request in inc order of enumeration

$R: \{R_1, \dots, R_m\}$ 1-to-1 functions $F: R \rightarrow N$, if req R_i , it can only req R_j with $F(R_i) < F(R_j)$

Or it must first release all Res R_i that $F(R_i) > F(R_j)$

10.5 DL Avoidance

each process declare max number of Res each type may need
algorithm dynamically examines the res-alloc state $\rightarrow$ never circular wait

Safe state: $\exists$ a safe sequence that all p can finish 1 by 1

No dl, not safe: may dl $\leftarrow$ DL

Banker's algorithm: multi ins of each Res, each p prior claim need max,

when p req, may wait; when p get all req, must return in finite time

#P = n, #Res = m, Available[m], Max[i, j] = k; P_i may req $\leq k$ of R_j

Allocation[i, j] = k, P_i allocated #k of R_j ; Need: Max-Allocation

1. Let Work and Finish be vectors of length m and n, respectively. Initialize:

Work = Available

Finish[i] = false for $i = 0, 1, \dots, n-1$

Safe?

2. Find an index i such that both:

(a) Finish[i] = false

(b) Need_i $\leq$ Work (i.e., for all k, Need[i, k] $\leq$ Work[k])

If no such i exists, go to step 4

3. Work = Work + Allocation;

Finish[i] = true

go to step 2

4. If Finish[i] == true for all i, then the system is in a safe state

Request = request vector for process P_i . If Request[i, j] = k then process P_i wants k instances of resource type R_j

1. If Request_i $\leq$ Need_i go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim

2. If Request_i $\leq$ Available, go to step 3. Otherwise, P_i must wait, since resources are not available

3. Pretend to allocate requested resources to P_i by modifying the state as follows:

Available = Available - Request;

Allocation_i = Allocation_i + Request_i;

Need_i = Need_i - Request_i;

● If safe $\Rightarrow$ the resources are allocated to P_i

● If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored

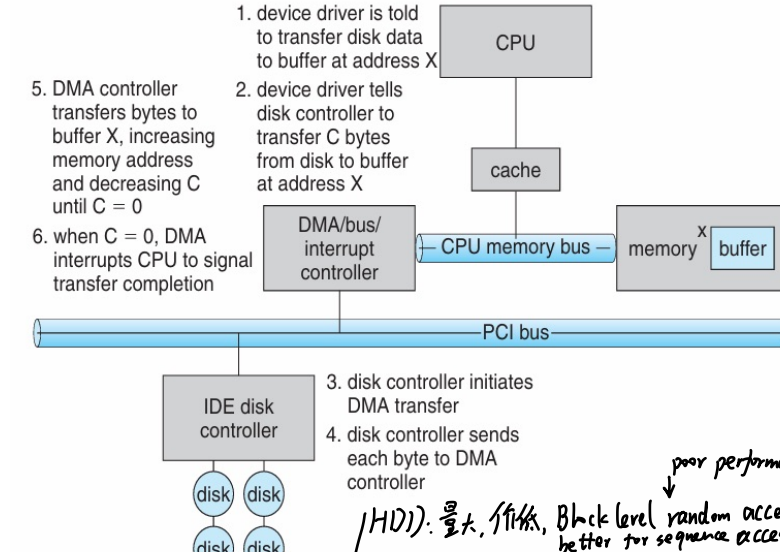
Res-req

Lecture 11 I/O and Storage.

11.1 CPU communicate with I/O device

- (1) Canonical View of devices: interface present $\rightarrow$ sys
- Status reg: current status of device: busy? ready? err?
- Command reg: tell device what to do: read/write?
- Data reg: pass/get data to/from device. $\rightarrow$ 一次可提交多个请求
- Modern NVMe SSD support multiple Command queues $\rightarrow$ parallelism
- (2) Basic I/O:
 - ① Polling: read status until ready; data $\rightarrow$ d reg; Command $\rightarrow$ c reg; wait $\rightarrow$ not busy (finished) inefficient and inconvenient; waste CPU time to wait for slow device; if switch, Data may be DW
 - ② Interrupt: OS issue I/O req, the put the calling p to sleep. CW to other process, when device finished, raise a hardware interrupt CPU jump to the OS at a predetermined interrupt handler.
 - (i) Hardware support: Interrupt-request line: a CPU wire trigger by I/O check by processor after each instruction, save state $\rightarrow$ interrupt handler
 - Interrupt controller: defer 延迟 during critical processing (pending), dispatch to correct handler (keyboard!), multi priority interrupts (high preempt low)
 - APIC x86, GIC ARM, PLIC (外部) CLINT (内部 timer...) RISC-V
 - (ii) Software support:
 - handler receive interrupt, Maskable to ignore or delay some 延迟 nonmaskable: e.g. unrecoverable mem err
 - Table of interrupt vectors, specify routine. (中断号 $\rightarrow$ handler addr)
 - 链式链上的中断依次执行, 不是自己发起的 return (trade off)
 - (3) Addr Method: (privileged ins)
 - Programmed I/O: exploit I/O ins, in/out on x86: out 0x21, AL
 - (main) Mem-Mapped I/O: reg/I/O appear in physical addr space
 - $\rightarrow$ I/O by load and store ins, protection with addr translation
 - PCIe BARs: expose device mem to CPU addr space.
 - Can be vector for attacks, modern use IOMMU to restrict

11.2 More efficient data movement: DMA (bypass CPU I/O $\leftrightarrow$ MEM)



11.3 Storage device

- (1) HDD: Magnetic disk: Sector 最小单位. ring of sec $\rightarrow$ track (1 μ m wide)
- stack of track $\rightarrow$ cylinder, head position on cylinders
- tracks are separated by guard regions, reduce likelihood neighboring tracks are corrupted
- Seek time: arm 移动到正确 track 上 4-8ms
- Rotational latency: 等待 sector 转到 head 下, avg = $\frac{T}{2}$ e.g. 8ms cycle $\Rightarrow$ avg 4ms
- Transfer time: File size/rate Disk Latency: Queue + Controller + Seek + Rotational + Transfer

request $\rightarrow$ software queue $\rightarrow$ Hardware Controller $\rightarrow$ Media time

random sector: Seek + Rot + Trans
random place, same cylinder: Rot + Trans
next sector, same track: Trans only

Lecture 12 File System

12.1 File System: Layer of OS that transforms block interface of disks into files, dirs ... Components:

- (1) Naming: interface to find files by name, not by blocks
- (2) Disk management: collecting disk blocks into files
- (3) Protection: Layers to keep data secure
- (4) Reliability (Durability): keeping of files durable despite crashes, media failure, attacks
- User: Durable data structure (inside OS): Collections of Blocks (logical trans unit)
- System: (sys call interface): Collections of Bytes, doesn't matter kind.
- Block size $\geq$ sector size 每次读写都按 block 操作

12.2 Directory: hierarchical structure - files, dirs (link to another)

each has a name and attributes, files have data, Links $\rightarrow$ DAG
Files: named permanent storage: data (blocks), Metadata (Attribute): Owner, size, RWX, Owner, Group, Other last opened

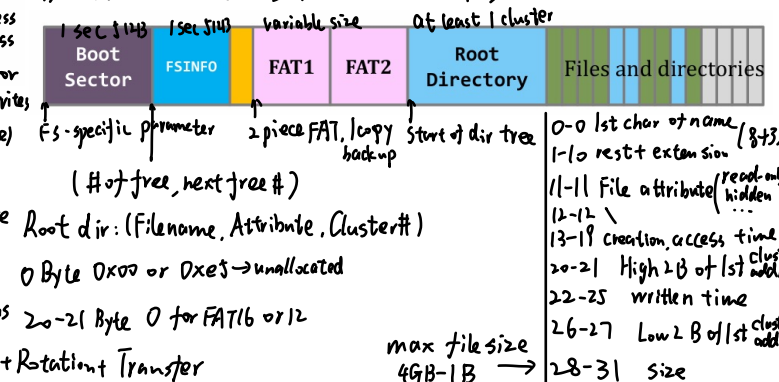
12.3 Disk management policies

Access disk as linear array of sectors $\rightarrow$ OS must deal with bad sector
- sector as vector (cylinder, surface, sector), sort in cylinder-major order,
- Logical Block Addressing: sector has integer addr from 0 $\rightarrow$ max # of sectors
Controller addr $\rightarrow$ physical position (hardware shields OS from structure of disk)
Need way to track free disk blocks: link free blocks, too slow
- Bitmap to represent free $\leftarrow$ 0

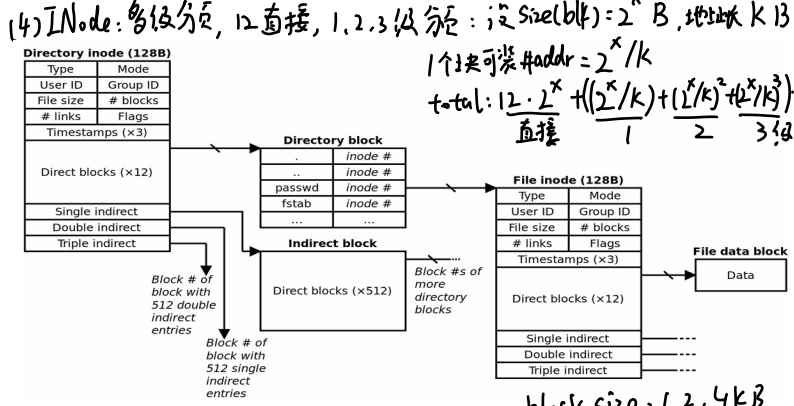
File Header: track which blocks belong at which offsets within logical file struct
- "translate" which part of file I want $\rightarrow$ "where to find at disk"
- 大多顺序集中使用: Optimize placement of files' disk blocks to match access and usage pattern.

12.4 Layout

- (1) Contiguous allocation: 连续分配, 记录 (name, start, addr, Size) CD only delete easy, but External Frag $\rightarrow$ hard to create new, write only read
- Defragmentation: slow on disk to move blocks together
- file can not grow, because it may have no space to expand.
- (2) Linked allocation: disk $\rightarrow$ 等大 block, 每 block 开头留 4B $\rightarrow$ next block
- 目录记录 (name, first block, size) $\rightarrow$ 方便 ls -l No external frag, easy growth
- but internal frag, file last block always no full
- poor random access performance, and single point of failure
- (3) FAT: File allocation table: centralize all the block links as FAT
FAT: (Block, next Block) next = 0 空块 next = -1 结束 / 12-12 bit
block bit $\rightarrow$ max # block $\rightarrow$ max disk size FAT { 16-16 bit, 32-28 bit
But MS $\rightarrow$ max 32GB, let we use NTFS



LFN: 11 Byte = 0x0F 1-10, 14-25, 28-31
 LFN n ... LFN 1 + Normal Entry →
 n ≤ 20
 FAT: 0 record: from root dir read name → cluster #; read n records; FAT find next
 Damage: if next = EOF, stop ② append: locate last cluster, FSINFO find space, write to find, update FATs, FSINFO, write update file size
 ③ Delete: FAT next cluster all 0, update FSINFO, Spatial locality, root dir the name 1st byte ⇒ 0x05



(5) Ext 2/3: Disk layout: for ext 2, 3 ⇒ FS size = 4.8, 16 TB
 FS → block groups, every block group same structure
 Super Block: GDT, Reserved, Blk bitmap, iNode bmap, iNode Table, Data blks
 FS specific data: location of bitmap, table free blk, iNode #
 e.g. total # of blks
 iNode bitmap → # of files in the FS is fixed, 有访问也不能再新建
 Why group? Performance: spatial, iNode, data blk related files together
 Reliability: superblk and GDT are replicated in each group
 create new at /dir/file.dat: 找 /dir, BG 0 → iNode 2 root, ...
 iNode Table look up, get block #, read to get new iNode #
 创建时查 iNode table, 找 1st 0 bit, 改为 1, 更新 Sup, GDT, root dir 指向
 空文件: iNode → 1, 加数据 blk → 1 (Len, Name, iNode #)

iNode Structure (128 bytes long)

Bytes	Value
0-1	File type and permission
2-3	User ID
4-7	Lower 32 bits of file sizes in bytes
8-23	Time information
24-25	Group ID
26-27	Link count (will discuss later)
...	...
40-87	12 direct data block pointers
88-91	Single indirect block pointer
92-95	Double indirect block pointer
96-99	Triple Indirect block pointer
...	...
108-111	Upper 32 bits of file sizes in bytes

Directory Entry in a dir blk

Bytes	Value
0-3	iNode number of that file/directory
4-5	Length of this entry
6-6	Length of the filename
7-7	File Type
8+	Name in ASCII (max 255 character)

12.5 Hard, Soft link
 Hard: Another filename → exist iNode → link count ++
 delete: unlink() → (C--, 删除) root dir row (iNode #, name) block, iNode deallocated
 (Symbolic) Soft: new iNode, store target pathname using 12+1+1 blk ptr space
 other wise / additional data blk if path < 60 chars (Fast symlink)
 when delete, link still; target move, soft invalid;

12.6 MMap: Mem-mapped file: 把文件当内存访问, 而不是 read(), write()
 Tradition I/O: multiple copies (kernel-user buffer), multiple sys call
 mmap: 先在 Page table 预留虚拟地址, VMA struct → mmap 加记录
 when page fault → direct disk → page cache (Data only store once, CPU doesn't do move between mem, zero copy, #read ↓)

Lecture 13 OS Security

13.1 Confinement: must run untrusted code, sys must "confine" it
 - restrict the interactions of a program to limit a set of resource
 - prevent unintended interference with other parts of the system
 (1) Isolation: boundary enforcement, ensure component operate independently
 ① physical: 在不同电脑上跑
 ② hardware: 降 privilege, within a process
 pre reg. PT config by OS only see its own ker U bit

(i) Isolation between privilege level: dual mode, user program iso
 (ii) Isolation within privilege level: intra-process compartmentalization, intra-kernel com...: eBPF
 Software fault isolation, hardware-based solution: MPK, MTE,

SFL: Mem Sandbox: 限制 code 访问内存区域; Control-Flow restriction: 限制 jump addr and \$0xf-e0.%eax ⇒ 所有合法代码入口在 32B 对齐; Ins-level: 有分支, int 0x80 (不执行) disabled
 MPK: + 4 bit key in PTE, reg: PKRU: 32 bit 16 pair key (access disabled, write disabled)
 user mode 可 hL reg EAX → PKRU (WRPKRU), PKRU → EAX (RDPKRU)
 Pros: 1. Changing acc rights of a page < 20 CPU cycle 2. Allow changing rights for up to 16 eBPF: safe, efficient VM in kernel, runs user-defined program without modifying kernel source or loading modules. Load: user space eBPF Byte code verify: kernel checks safety. Attach: program hook event. Run: JIT-compiled when event trigger → kernel

(2) Access Control: restrict unauthorized users and sys from acc res.
 Subject: users/process, Obj: files/device, Permission: RWX, Capability: <obj, per> on sub
 Implementation: Access Control matrix $\begin{matrix} & \text{File} & \text{File} \\ & \text{P}_1 & \text{P}_2 \\ \text{U} & \text{w} & \text{x} \end{matrix}$, ACL: attach to each obj. <subject, permission>
 Linux AC: Discretionary Access Control: user/group level control, per set by owner of res
 Linux chmod, chown. Mandatory AC: central authority, security policy
 each file/user → security label, security framework (SELinux)

Capability: held by subject, Issuance → Access request → Validation → action
 Unforgable: crypto/hardware enforced. Deletable: shared/revoked dynamically
 Self-Describing: contain <obj, per> No Global check: only check locally
 (3) Trusted exe env: ① CPU isolation: TEE external 软件不能访问 TEE 中 mem reg cache
 ② Memory encryption: Pmem used is 加密 on the fly, keys dynamic generated by TEE 硬件
 ③ Remote attestation: can ensure integrity of initial state of software running in TEE

13.2 Sys Hardening: (1) Vulnerability Discovery ① Program Analysis (i) Static data flow, static taint, symbolic exe (ii) Dynamic: test running: Fuzzing (iii) hybrid: Concolic exe
 (2) Fuzzing: inject unexpected input. Input generation: Mutation/generate
 Feedback: Coverage-Guided, black/grey box Target knowledge: White/black box / hybrid
 (3) Exploit prevention: ASLR: 随机地址; Stack canary: buffer OF, random value placed before return addr, 查 canary 是否被修改; Control Flow Integrity: restrict program exe to valid control-flow paths, preventing exploits like Return-Oriented/Jump-Oriented Programming, Call/return hijacking

13.3 Detection (1) malware detection: viruses, worms, trojans, ransomware, spyware, adware, rootkits (2) technique: Signature, Heuristic, Behavioral, ML, Sandbox
 (2) Host-based intrusion detection: monitor single host to detect anomaly potential intrusions/malicious activity, event sensor, analyzer, response mechanism

Domain	Technologies/Techniques	Threats Mitigated
Perimeter Security	Firewalls, IDS/IPS, NAC	Unauthorized access, intrusions
Data Protection	TLS/SSL, VPNs, Data Loss Prevention (DLP)	Eavesdropping, man-in-the-middle
Access Control	Zero Trust, MFA, IAM	Credential theft, insider threats
Threat Monitoring	SIEM, NDR (Network Detection & Response)	APTs, lateral movement
Wireless Security	WPA3, 802.1X, Rogue AP detection	Wi-Fi spoofing, Evil Twin attacks
Cloud/Edge Security	SASE, CASB, Zero Trust Network Access (ZTNA)	Misconfigurations, cloud breaches